Suivi de flotte de véhicules

Projet Big Data • ISSE-05

MQTT · Kafka · MinIO · Hudi · Spark · Airflow · Streamlit

Équipe de 4 étudiants : noms à compléter

Pourquoi suivre une flotte ?

Localiser les véhicules sur une carte.

Repérer les excès de vitesse et les sorties de zone.

Mesurer distance et vitesse par véhicule et par jour.

Identifier les zones où les véhicules s’arrêtent.

Le cadre du projet

Source GPS simulée autorisée par le sujet.

MQTT → Kafka, topic « positions ».

Bronze brut → Hudi Silver → Hudi Gold.

Spark Streaming, SQL et MLlib ; Airflow quotidien.

Une commande Docker Compose et une carte Streamlit.

Du GPS au tableau de bord

GPS simulés → Mosquitto → passerelle Python → Kafka

Kafka → Spark Structured Streaming → Bronze + Hudi Silver

Airflow → Spark SQL + KMeans → Hudi Gold

Exports Gold sur MinIO → carte et indicateurs Streamlit

Airflow → contrôle qualité + rapport quotidien

Un flux reproductible

12 véhicules simulés autour de Casablanca.

Un message par véhicule toutes les 2 secondes.

event_id, vehicle_id, event_ms, latitude, longitude, speed_kmh.

Arrêts, vitesses variables et véhicule sortant de la zone.

Horodatage UTC ; position géographique synthétique.

MQTT et Kafka

MQTT QoS 1 : transport léger pour les boîtiers.

Passerelle : conservation du payload et de l’UUID.

Kafka : topic positions, 3 partitions.

Clé de partition : vehicle_id.

Des doublons sont possibles ; Hudi les absorbe par clé.

Bronze, Silver, Gold

Bronze : payload brut + topic, partition et offset Kafka.

Silver : positions validées, jour UTC et indicateurs d’alerte.

Gold : bilan journalier, dernière position et incidents.

Hudi Copy-on-Write : upserts, clés stables et commits.

MinIO : données, checkpoints, modèles et rapports.

Détecter et mesurer

Excès de vitesse : speed_kmh > 80.

Sortie de zone : rectangle autorisé autour de Casablanca.

Distance : segments de Haversine ordonnés par véhicule.

Les trous GPS de plus de 5 minutes sont ignorés.

SQL : distance, vitesse moyenne/max et mesures en incident.

Découvrir les zones d’arrêt

Entrée : positions à vitesse ≤ 5 km/h.

Projection locale approximative des coordonnées en km.

MLlib KMeans : 2 à 3 centres, seed = 42.

Réentraînement via le DAG refresh_gold.

Résultat : centres sur une carte ; pas de score de précision inventé.

Airflow pilote les calculs

refresh_gold : toutes les 2 minutes.

Appel HTTP interne → calcul Spark → export Gold.

Un seul calcul à la fois, tentatives en cas d’échec.

daily_driving_report : fraîcheur, qualité et bilan UTC.

Rapport JSON conservé dans MinIO.

La vue du gestionnaire

Carte : dernière position de chaque véhicule.

KPI : flotte, distance échantillonnée, mesures d’excès.

Tableaux : bilan journalier et derniers incidents.

Carte des centres d’arrêt KMeans.

Export CSV et heure du dernier traitement.

Ce qui est vérifié

Tests unitaires : distance, qualité et seuils d’alerte.

Validation de la configuration Docker Compose.

Compilation Python et upsert Hudi local réussis.

Recette Spark réussie : transformations, SQL et KMeans.

Démarrage Docker complet : à valider sur le PC.

Une preuve à chaque couche

1. docker compose up --build -d
2. Consommer trois messages du topic positions.
3. Montrer Bronze et les tables Hudi sur MinIO.
4. Montrer un DAG réussi dans Airflow.
5. Présenter carte, incidents, centres et rapport.

Quatre responsabilités, une chaîne

Membre 1 : simulation GPS, MQTT, Kafka.

Membre 2 : MinIO, Bronze, Hudi et qualité.

Membre 3 : transformations Spark, SQL, KMeans.

Membre 4 : Airflow, Streamlit et restitution.

Intégration, tests et Docker : travail collectif.

Une base à valider et enrichir

Chaîne conforme aux technologies demandées dans ISSE-05.

Données simulées et règles explicables pour apprendre.

Limites : un broker, relecture complète, exports au driver.

Suite : vraie flotte, fenêtres incrémentales, sécurité, évaluation ML.

Avant soutenance : compléter les résultats et répéter la démo.